

# **Algorithms for Arithmetic**

# Algorithms:

*Simple definition:*

Well-defined procedure for carrying out some task!

*Desirable Properties:*

- Guaranteed to conclude /halt
- Accuracy, provides the correct outcome
- Efficiency: time, operations, memory, machines (parallel)

Example of simple algorithms?

- Arithmetic! – Addition, Multiplication

# Algorithms for Arithmetic

Input description?

- Arabic numeral system! [just 10 symbols]
- Discovered in India around or before 600AD
- Al Khwarizmi in the Arab world and Fibonacci's work popularized it in Europe

Al Khwarizmi – basic procedures for adding, multiplying & dividing... square root calculation and representing  $\pi$  as an Arabic number (in digits)

- Precise, unambiguous, mechanical, efficient, correct procedures... that is *Algorithms* (coined in Europe)!

*Algorithm for addition of  $X+Y$  ( $X, Y$  are string of base-10 digits):*

**Algorithm 1 – Counting (single digits) on hands!**

- Start at  $X$  and increment by  $Y$

Efficiency (time)?

- If  $X$  and  $Y$  are  $n$  ( $\leq n$ ) digits each:

Time taken to increment  $X$  by 1 is  $\leq n$  !

Time taken to increment  $X$  by  $Y \leq Y \cdot n \leq (10^n - 1)n$

... that can be huge!

↓  
If no carries, then can be less

**Algorithm 2 – Right to Left Pass Sum with Carry**

Look up

3 7 9 1  
+ 8 1 2 5  
-----

11 9 1 6

Time taken to increment  $X$  by  $Y$  is  $n$

Linear in  $n$ !

Can be  $n + 1$  digits

↙  
 $c_0 + c_1 n$

- Addition cannot be faster as reading /writing  $n$  digits require  $n$  steps!

We are discussing Base 10 representing numbers up to  $10^n - 1$

Base  $b$  can represent  $N$  in  $\lceil \log_b(N + 1) \rceil$  digits! ( $N$  in decimal)

- Why not base large value: Number of symbols will be more, and therefore, you will need larger look up table for adding 2 numbers increasing memory requirement!

Varying bases are not practical as well because code should not change with number of digits!

*Algorithm for Multiplying  $X \times Y$  ( $X, Y$  are base-10  $n$  digit numbers):*

**Algorithm 1 – Add  $X$  to itself  $Y$  times**

Time taken to add  $X$  by  $X$  is  $n$

Time taken to add  $X$  by  $X, Y$  times =  $Y \cdot n \leq (10^n - 1)n$

## Algorithm 2 – Right to Left Pass Product (& sum) with Carry

Multiplication Table!

|             |   |                                                        |
|-------------|---|--------------------------------------------------------|
| 3 7 9 1     | } | At most $(C_1n - C_2) \times n$ time!                  |
| x 1 2 5     | } |                                                        |
| 1 8 9 5 5   |   |                                                        |
| 7 5 8 2     | } | plus<br>At most $(C_3n + C_4) \times (n - 1)$<br>time! |
| 3 7 9 1     | } |                                                        |
| 4 7 3 8 7 5 |   |                                                        |

Can be  $2n$  digits

So, time taken to multiply  $X$  by  $Y$  is  $\leq Cn^2 + \dots$ . [ $O(n^2)$ ]

Can multiplication be faster as reading /writing  $n$  ( $2n$ ) digits only require  $n$  ( $2n$ ) steps?

Well, yes you can!

But before that let's see another approach by Al Khwarizmi

### Algorithm 3 – Al Khwarizmi's approach

Consider  $11 \times 13$ :

|              |     |              |           |   |   |   |   |   |                                |
|--------------|-----|--------------|-----------|---|---|---|---|---|--------------------------------|
| 11           | 13  |              | In binary | → | 1 | 1 | 0 | 1 |                                |
| 5            | 26  |              | →         | × | 1 | 0 | 1 | 1 |                                |
| 2            | 52  | (strike out) |           |   |   |   |   |   |                                |
| 1            | 104 |              | ↔         |   |   |   |   |   |                                |
| 143 (answer) |     |              |           |   | 1 | 1 | 0 | 1 | (1101 times 1)                 |
|              |     |              |           |   | 1 | 1 | 0 | 1 | (1101 times 1, shifted once)   |
|              |     |              |           |   | 0 | 0 | 0 | 0 | (1101 times 0, shifted twice)  |
|              |     |              |           |   | + | 1 | 1 | 0 | 1                              |
|              |     |              |           |   | 1 | 0 | 0 | 0 | 1                              |
|              |     |              |           |   |   |   |   |   | (1101 times 1, shifted thrice) |
|              |     |              |           |   | 1 | 0 | 0 | 0 | 1                              |
|              |     |              |           |   | 1 | 1 | 1 | 1 | (binary 143)                   |

$m$  bits

$$\text{Time: } \leq Cm^2 = Cn^2(\log_{10} 2)^{-2} [O(n^2)]$$

We can repackage the above into this recursive procedure:

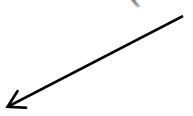
$$x \cdot y = \begin{cases} 2(x \cdot \lfloor y/2 \rfloor) & \text{if } y \text{ is even} \\ x + 2(x \cdot \lfloor y/2 \rfloor) & \text{if } y \text{ is odd.} \end{cases}$$

# Algorithm 4 – Multiplication by Divide & Conquer!

Integer 

$$(a + bi)(c + di) = ac - bd + (bc + ad)i$$

By Gauss:

$$bc + ad = (a + b)(c + d) - ac - bd$$


Now-

$$X = \begin{array}{cccc} 3 & 7 & 9 & 1 \\ \downarrow & & \downarrow & \\ X_h & & X_l & \end{array}$$

$$Y = \begin{array}{cccc} 0 & 1 & 2 & 5 \\ \downarrow & & \downarrow & \\ Y_h & & Y_l & \end{array}$$

$n \rightarrow$  is even or  
made even!

$$X = X_h 10^{n/2} + X_l \quad Y = Y_h 10^{n/2} + Y_l$$

So:

$$\begin{aligned} X \times Y &= \left( X_h 10^{\frac{n}{2}} + X_l \right) \left( Y_h 10^{\frac{n}{2}} + Y_l \right) \\ &= X_h Y_h 10^n + (X_h Y_l + X_l Y_h) 10^{\frac{n}{2}} + X_l Y_l \end{aligned}$$

Recursion!

4 times  $n/2$  digits  
multiplication for 1  $n$  digit  
multiplication



Base case: 1 digit look up